

Competitive Programming

Section 64: Multiplicative Number Theory

Competitive Programming Course

2026-08-20

Table of contents

1	From formulas about one integer to transforms over all divisors	3
2	Precise vocabulary	4
3	One linear sieve for SPF, totient, and Möbius	5
3.1	Why every composite is generated exactly once	5
3.2	Deriving the totient updates	6
3.3	Deriving the Möbius updates	6
3.4	Compact C++ implementation	7
4	Pattern 1: replace a gcd value by divisor contributions	9
4.1	Tiny trace	9
5	Pattern 2: group equal floor quotients	10
6	Deriving the summatory-totient recurrence	11
7	Pattern 3: count cumulative LCM information, then invert	12
8	Pattern 4: turn a counting oracle into an order statistic	13
9	Proof and implementation habits	14
10	Common wrong approaches	15
11	Exercises	16
12	Before submitting	17

1 From formulas about one integer to transforms over all divisors

Earlier sections factorized individual values and used inclusion-exclusion on their prime factors. Master-level problems often reverse the scale: all values up to M matter, and the desired quantity is hidden behind a sum over divisors. The decisive question becomes:

Do I know cumulative information for every multiple/divisor, while the statement asks for the exact value at one gcd or lcm?

If yes, think of a transform on the divisibility relation.

2 Precise vocabulary

For arrays indexed by positive integers, the **divisor zeta transform** is

$$Z[m] = \text{sum of } F[d] \text{ over every } d \text{ dividing } m.$$

Its inverse recovers exact values in increasing order:

$$F[m] = Z[m] - \text{sum of } F[d] \text{ over proper divisors } d \text{ of } m.$$

The dual form sums over multiples instead. Both can be evaluated by sieve-like loops. This is ordinary inclusion-exclusion organized by the divisor lattice.

The **Dirichlet convolution** of arithmetic functions f and g is

$$(f * g)(n) = \text{sum over } d \mid n \text{ of } f(d) * g(n / d).$$

The constant-one function has inverse μ , the Mobius function. Therefore a divisor sum can be inverted algebraically with μ , or algorithmically by subtracting already recovered proper-divisor contributions.

3 One linear sieve for SPF, totient, and Möbius

Section 31 constructed an SPF table by visiting multiples of every prime. That is simple and usually fast enough. A **linear sieve** organizes the work more precisely: every composite number is generated exactly once, together with its smallest prime factor.

Maintain the primes found so far. When the outer loop is at x , try products $x \cdot p$ in increasing prime order, but stop after $p = \text{spf}[x]$.

```
spf[1] = 1
phi[1] = 1
mu[1] = 1

for x = 2..N:
    if spf[x] == 0:
        x is prime
        spf[x] = x
        phi[x] = x - 1
        mu[x] = -1
        append x to primes

    for p in primes:
        if x*p > N:
            break
        spf[x*p] = p

        if p == spf[x]:
            phi[x*p] = phi[x] * p
            mu[x*p] = 0
            break
        else:
            phi[x*p] = phi[x] * (p - 1)
            mu[x*p] = -mu[x]
```

3.1 Why every composite is generated exactly once

Take a composite y and let $p = \text{spf}[y]$. Write $y = x \cdot p$. Every prime factor of x is at least p , so $p \leq \text{spf}[x]$. Therefore the inner loop for this particular x reaches p before it stops and generates y .

Could another pair $x' \cdot q$ generate the same y ? The sieve records a product only while $q \leq \text{spf}[x']$. That condition makes q the smallest prime factor of $x' \cdot q$. Hence q would have to equal $\text{spf}[y] = p$, forcing $x' = y/p = x$. Existence and uniqueness together prove that the total number of successful inner-loop iterations is at most $N-1$. This is the source of the $O(N)$ bound.

The `break` in the equal-prime case is essential. Once $p = \text{spf}[x]$, any later prime $q > p$ would make $x*q$ have smallest prime factor p , so that product belongs to the future representation $(x*q/p)*p$, not to the current x .

3.2 Deriving the totient updates

Suppose p is the prime used to form $x*p$.

If p already divides x , multiplying by one more p increases the prime power but introduces no new forbidden residue fraction:

$$\text{phi}(x*p) = p * \text{phi}(x).$$

For example, going from 12 to 24 doubles both the modulus and the number of coprime residues.

If p does not divide x , then x and p are coprime. The totient function is multiplicative on coprime arguments, and $\text{phi}(p)=p-1$, so

$$\begin{aligned}\text{phi}(x*p) &= \text{phi}(x) * \text{phi}(p) \\ &= \text{phi}(x) * (p-1).\end{aligned}$$

This is why testing $p == \text{spf}[x]$ is sufficient: primes are processed in increasing order, so equality means p divides x ; reaching a smaller p means it does not.

3.3 Deriving the Möbius updates

The Möbius function is determined by the prime factorization:

$$\begin{aligned}\mu(1) &= 1 \\ \mu(n) &= 0 && \text{if some prime square divides } n \\ \mu(n) &= (-1)^k && \text{if } n \text{ is a product of } k \text{ distinct primes.}\end{aligned}$$

If p already divides x , then $x*p$ contains p^2 , so $\mu[x*p]=0$. Otherwise one new distinct prime factor is introduced, which flips the sign:

$$\mu[x*p] = -\mu[x].$$

For $N=10$, the arrays begin as follows:

x	factorization	$\text{spf}[x]$	$\text{phi}[x]$	$\mu[x]$
1	1	1	1	1
2	2	2	1	-1
3	3	3	2	-1
4	2^2	2	2	0
5	5	5	4	-1
6	$2 \cdot 3$	2	2	1

x	factorization	spf[x]	phi[x]	mu[x]
8	2^3	2	4	0
10	$2 \cdot 5$	2	4	1

3.4 Compact C++ implementation

```

struct LinearSieve {
    vector<int> primes, spf, phi, mu;

    explicit LinearSieve(int n)
        : spf(n + 1), phi(n + 1), mu(n + 1) {
        if (n >= 1) spf[1] = phi[1] = mu[1] = 1;

        for (int x = 2; x <= n; ++x) {
            if (spf[x] == 0) {
                spf[x] = x;
                phi[x] = x - 1;
                mu[x] = -1;
                primes.push_back(x);
            }
            for (int p : primes) {
                if (1LL * x * p > n) break;
                spf[x * p] = p;
                if (p == spf[x]) {
                    phi[x * p] = phi[x] * p;
                    mu[x * p] = 0;
                    break;
                }
                phi[x * p] = phi[x] * (p - 1);
                mu[x * p] = -mu[x];
            }
        }
    }
};

```

The same recurrence in Python is almost identical. The inner loop is still linear in aggregate; it is not a nested $O(N * \text{number_of_primes})$ loop.

```

def linear_sieve(n):
    primes = []
    spf = [0] * (n + 1)
    phi = [0] * (n + 1)
    mu = [0] * (n + 1)
    if n >= 1:
        spf[1] = phi[1] = mu[1] = 1

    for x in range(2, n + 1):
        if spf[x] == 0:

```

```

    spf[x] = x
    phi[x] = x - 1
    mu[x] = -1
    primes.append(x)

for p in primes:
    if x * p > n:
        break
    spf[x * p] = p
    if p == spf[x]:
        phi[x * p] = phi[x] * p
        mu[x * p] = 0
        break
    phi[x * p] = phi[x] * (p - 1)
    mu[x * p] = -mu[x]

return primes, spf, phi, mu

```

Use the ordinary SPF sieve when simplicity matters more than the constant factor. Use the linear version when several multiplicative functions are needed together, or when the proof that every composite is processed once is useful for extending the sieve with another recurrence.

4 Pattern 1: replace a gcd value by divisor contributions

Euler's identity says:

```
g = sum(phi(d) for d dividing g).
```

For a fixed positive exponent k , the analogous Jordan function J_k is defined by

```
g^k = sum(J_k(d) for d dividing g).
```

You do not need its prime-product formula. Compute $\text{value}[g] = g^k$, then invert the divisor sums:

```
J = value
for d = 1..M:
    for multiple = 2*d, 3*d, ...:
        J[multiple] -= J[d]
```

Now sum over pairs. A pair contributes $J_k(d)$ exactly when both values are divisible by d . If $\text{cnt}[d]$ values are divisible by d , there are $\text{cnt}[d] \text{ choose } 2$ such pairs. Hence

```
sum over pairs gcd(ai, aj)^k
= sum over d J_k(d) * C(cnt[d], 2).
```

This change of summation is the main move: count pairs for a divisibility condition, not separately for every possible gcd.

4.1 Tiny trace

For values 2, 4, 6 and $k = 1$, the divisible counts are

d	cnt[d]	pairs divisible by d	phi(d) contribution
1	3	3	3
2	3	3	3
3	1	0	0
4	1	0	0

The total is 6, equal to $\text{gcd}(2,4)+\text{gcd}(2,6)+\text{gcd}(4,6)$.

5 Pattern 2: group equal floor quotients

Loops containing `floor(n / i)` often look linear, but this quotient has only about $2\sqrt{n}$ distinct values. If `q = floor(n/l)`, the last index with the same quotient is

```
r = floor(n / q).
```

Thus process the whole interval `[l,r]`, then set `l = r+1`.

Example for `n=20`:

```
i interval: 1 | 2 | 3 | 4 | 5 | 6 | 7..10 | 11..20
n//i:      20 |10 | 6 | 5 | 4 | 3 |    2 |    1
```

This is the harmonic-block pattern. It is the bridge from a recurrence that mentions every `i` to a sublinear set of recursive arguments.

6 Deriving the summatory-totient recurrence

Let $\Phi(n) = \phi(1) + \dots + \phi(n)$. From

$$\sum_{d \mid m} \phi(d) = m$$

sum both sides for all $m \leq n$. A fixed d appears once for each multiple of d , namely $\lfloor n/d \rfloor$ times:

$$\sum_{d \leq n} \phi(d) * \lfloor n/d \rfloor = 1 + 2 + \dots + n.$$

The $d=1$ term is not the useful isolation. Regroup indices by the value of $\lfloor n/d \rfloor$ and express blocks through differences of Φ . After telescoping, the standard memoized form is:

```
Phi(n) = n(n+1)/2
for l = 2 while l <= n:
    q = n // l
    r = n // q
    Phi(n) -= (r-l+1) * Phi(q)
    l = r+1
```

Precompute small Φ ; memoize the larger arguments. Every recursive argument is smaller than n , and quotient grouping prevents a linear loop per state.

7 Pattern 3: count cumulative LCM information, then invert

Suppose `inside[m]` is the number of input values dividing `m`. Every pair chosen among these values has lcm dividing `m`, so

```
all[m] = C(inside[m], 2)
        = sum(exact[d] for d dividing m),
```

where `exact[d]` counts pairs with lcm exactly `d`. Divisor inversion recovers `exact` in increasing order. Notice the proof obligation: “both values divide `m`” is equivalent to “their lcm divides `m`.” Without that equivalence, the transform would count the wrong family.

8 Pattern 4: turn a counting oracle into an order statistic

The indicator of square-freeness has the identity

```
is_squarefree(x) = sum(mu(d) for d*d dividing x).
```

Summing it for $x \leq X$ and swapping the sums gives

```
count_squarefree(X)
  = sum from d=1 to floor(sqrt(X)) of mu(d) * floor(X / (d*d)).
```

This count is monotone in X , so binary search finds the smallest X whose count reaches k . This pattern is broader than square-free numbers:

```
fast prefix-count oracle + monotonicity = k-th object by binary search.
```

9 Proof and implementation habits

Before coding a divisor transform, write four meanings:

1. What does the original frequency array count?
2. What does the transformed cumulative array count?
3. What exact family does inversion recover?
4. Is the loop over divisors or over multiples?

Use signed 64-bit integers for indices up to `1e11`, and use a wider temporary or modular multiplication discipline when products can exceed 64 bits. In Python, integer width is safe but loop count is not; quotient grouping matters equally.

10 Common wrong approaches

- Factoring every pair: quadratic before factorization even starts.
- Writing `for i in range(1,n+1)` inside a large summatory query.
- Applying Mobius signs without first stating the cumulative family.
- Mixing “gcd is divisible by d” with “gcd divides d.”
- Binary-searching an order statistic with a prefix counter that is not proven monotone.

11 Exercises

A–C are guided applications of the three transforms. D is blind: its statement contains no algorithmic vocabulary. For D, first build a brute prefix counter, then ask which part can be replaced by an arithmetic identity.

12 Before submitting

- Test $M=1$, repeated ones, and all values equal.
- Normalize every subtraction modulo $1,000,000,007$.
- In a quotient-block loop, verify that $r \geq 1$ and the next 1 is $r+1$.
- In a square-root bound, check $d*d \leq X$ using a safe integer type.
- Compare every optimized routine with brute force below 50 .